

Resolucion del Practico 7

Testing de Software 2024

Índice

Introduccion	2
1. Ejercicio 1: NodeCachingLinkedList	2
1.1. Estructura del objeto	2
1.2. (a) Implementacion de <code>repOK()</code>	3
1.3. (b) Inicializacion de <code>maximumCacheSize</code>	4
1.4. (c) Propiedades <code>jqwik</code>	5
1.5. Generadores	6
2. Ejercicio 2: Point	7
2.1. Contrato <code>equals/hashCode</code>	7
2.2. Implementacion	7
2.3. Propiedad 1: contrato <code>equals/hashCode</code>	8
2.4. Propiedad 2: distancia sobre recta horizontal	8
3. Ejercicio 3: Date	9
3.1. Constructor y <code>repOk()</code>	9
3.2. <code>addDays(when, days)</code>	10
3.3. Propiedad: <code>addDays</code> siempre devuelve una fecha valida	11
Conclusiones	12

Introduccion

Este documento contiene la resolucion del *Practico 7* de Testing de Software, centrado en **Property-Based Testing (PBT)** usando la libreria `jqwik` sobre Java. En lugar de escribir tests con valores concretos, se enuncian *propiedades* que deben valer para *todas* las entradas dentro de un dominio razonable, y el framework las verifica generando cientos de instancias aleatorias.

A lo largo del practico se trabajan tres ejercicios:

- **Ejercicio 1. NodeCachingLinkedList:** una lista doblemente enlazada y circular con un *pool* de nodos cacheados. Hay que completar `repOk()` con todos los invariantes de representacion y escribir tres propiedades `jqwik` que validen el comportamiento.
- **Ejercicio 2. Point:** clase de puntos con coordenadas `float`. Se implementan `equals/hashCode` de manera consistente con el contrato de `Object`, se escribe un generador `@Provide` para puntos y dos propiedades: el contrato `equals/hashCode` y la distancia entre puntos sobre una recta paralela al eje X.
- **Ejercicio 3. Date:** representacion de fechas a partir de 1900. Se completa el constructor, `repOk()` y el metodo `addDays`, y se enuncia la propiedad *toda fecha valida sumada a un numero no negativo de dias devuelve una fecha valida*.

Convenciones. A lo largo del documento se usa la notacion estandar de `jqwik`: `@Property` marca metodos como propiedades, `@ForAll` parametros sobre los que `jqwik` muestrea valores, y `@Provide` define generadores. Cada propiedad se ejecuta con `tries = 200` o 250 iteraciones segun el caso, equilibrando cobertura y tiempo de ejecucion.

Como se corren los tests. Desde `tp7/assignmnet-7-rodeghiero`:

```
JAVA_HOME=$(/usr/libexec/java_home -v 17) mvn -Dmaven.repo.local
=.m2 \
-Djacoco.skip=true test
```

La flag `-Djacoco.skip=true` se usa porque el plugin de cobertura del template (0.8.2) es incompatible con la JVM mas reciente; PBT no necesita ese instrumento.

1. Ejercicio 1: NodeCachingLinkedList

1.1. Estructura del objeto

La clase mantiene dos sub-estructuras que conviven:

- **Lista principal:** doblemente enlazada y *circular*, con un nodo centinela `header`. Cada nodo `n` cumple `n.next.previous = n` y `n.previous.next = n`.
- **Cache:** una lista *simplemente* enlazada, accesible por `firstCachedNode`, que mantiene nodos reciclados. Cuando se remueve un elemento, el nodo se “limpia” (`previous = null`, `value = null`) y se devuelve a la cache hasta un maximo de `maximumCacheSize`.

1.2. (a) Implementacion de `repOK()`

Los invariantes son los siguientes, agrupados por costo creciente de chequeo:

- Validez basica: `header != null`, `header.next != null`, `header.previous != null`.
- Consistencia de tamanos: `size >= 0`, `cacheSize >= 0`, `maximumCacheSize >= 0` y `cacheSize <= maximumCacheSize`.
- Constante: `DEFAULT_MAXIMUM_CACHE_SIZE == 20`, como pide el enunciado.
- Lista principal: recorrer desde `header` siguiendo `next` hasta volver al centinela. En cada paso verificar los enlaces dobles y, con un `HashSet`, que no haya ciclos prematuros. Al final, `size` debe coincidir con el numero de nodos visitados menos uno (por el centinela).
- Cache: recorrer desde `firstCachedNode` siguiendo `next` hasta `null`. Cada nodo cacheado debe tener `previous == null` y `value == null`, no formar ciclos y *no* pertenecer tambien a la lista principal (se compara contra el `HashSet` anterior).
- Al final `cacheSize` debe coincidir con la cantidad de nodos recorridos en la cache.

Resumen del codigo.

```
public boolean repOK() {
    if (header == null) return false;
    if (header.getNext() == null || header.getPrevious() == null)
        return false;
    if (size < 0) return false;
    if (maximumCacheSize < 0 || cacheSize < 0 ||
        cacheSize > maximumCacheSize) return false;
    if (DEFAULT_MAXIMUM_CACHE_SIZE != 20) return false;

    Set<LinkedListNode> listNodes = new HashSet<>();
    LinkedListNode current = header;
    int listCount = 0;
    while (true) {
        if (current == null) return false;
        if (!listNodes.add(current)) return false;
```

```

        if (current.getNext() == null || current.getPrevious() ==
            null) return false;
        if (current.getNext().getPrevious() != current) return
            false;
        if (current.getPrevious().getNext() != current) return
            false;
        listCount++;
        current = current.getNext();
        if (current == header) break;
    }
    if (size != listCount - 1) return false;

    Set<LinkedListNode> cacheNodes = new HashSet<>();
    current = firstCachedNode;
    int cacheCount = 0;
    while (current != null) {
        if (!cacheNodes.add(current)) return false;
        if (listNodes.contains(current)) return false;
        if (current.getPrevious() != null) return false;
        if (current.getValue() != null) return false;
        cacheCount++;
        current = current.getNext();
    }
    return cacheSize == cacheCount;
}

```

1.3. (b) Inicializacion de maximumCacheSize

El constructor original dejaba `maximumCacheSize` en 0, lo cual desactiva la cache (porque `isCacheFull()` devuelve siempre `true` y ningun nodo se cachea). Eso es inconsistente con el nombre de la clase. Se corrige asignando `maximumCacheSize = DEFAULT_MAXIMUM_CACHE_SIZE` en el constructor:

```

public NodeCachingLinkedList() {
    header = new LinkedListNode();
    header.setValue(null);
    header.setNext(header);
    header.setPrevious(header);
    maximumCacheSize = DEFAULT_MAXIMUM_CACHE_SIZE;    // <--
    agregado
}

```

1.4. (c) Propiedades jqwik

Propiedad 1: al remover, la cache crece en uno.

```
@Property(tries = 200)
void luegoDeRemoverUnElementoSeIncrementaEnUnoElTamanoDeCache(
    @ForAll("escenariosRemocion") EscenarioRemocion escenario) {
    NodeCachingLinkedList ncl = escenario.lista;
    assertTrue(ncl.repOK());
    int cacheAntes = ncl.getCacheSize();

    Integer removido = ncl.removeIndex(escenario.index);

    assertNotNull(removido);
    assertEquals(cacheAntes + 1, ncl.getCacheSize());
}
```

El generador construye una lista de entre 1 y 15 elementos. Como ese rango es menor a `maximumCacheSize = 20`, la cache nunca llega a saturarse y la propiedad “cada remocion suma 1 al `cacheSize`” vale siempre.

Propiedad 2: con cache no vacia, `size + cacheSize` se conserva en `addFirst`.

```
@Property(tries = 200)
void siLaCacheNoEstaVacíaAgregarMantieneConstanteLaSumaDeNodos(
    @ForAll("escenariosAgregarConCacheNoVacía")
        EscenarioAgregarConCache escenario) {
    NodeCachingLinkedList ncl = escenario.lista;
    assertTrue(ncl.repOK());
    assertTrue(ncl.getCacheSize() > 0);
    int sumaAntes = ncl.getSize() + ncl.getCacheSize();

    ncl.addFirst(escenario.nuevoValor);

    int sumaDespues = ncl.getSize() + ncl.getCacheSize();
    assertEquals(sumaAntes, sumaDespues);
}
```

Esta propiedad captura la idea central de la cache: si hay nodos disponibles, `addFirst` *reutiliza* un nodo en lugar de crear uno. Por lo tanto, `size` aumenta en 1 y `cacheSize` disminuye en 1, manteniendo constante la suma. Para garantizar la precondition (cache no vacia), el generador primero construye una lista y despues remueve un elemento.

Propiedad 3: `repOK()` se preserva por `removeIndex`.

```
@Property(tries = 200)
void eliminarUnElementoMantieneElInvarianteDeRepresentacion(
    @ForAll("escenariosRemocion") EscenarioRemocion escenario) {
```

```

NodeCachingLinkedList ncl = escenario.lista;
assertTrue(ncl.repOK());
ncl.removeIndex(escenario.index);
assertTrue(ncl.repOK());
}

```

Esta es la propiedad clásica de un invariante de representación: cualquier operación pública sobre un objeto válido debe dejarlo en estado válido. Si alguna manipulación interna (manejo de enlaces, paso del nodo a la cache, decremento de `size`) rompiera la coherencia, el segundo `repOK()` la detectaría.

1.5. Generadores

Los `@Provide` construyen escenarios completos para que cada propiedad reciba exactamente lo que necesita:

```

@Provide
Arbitrary<EscenarioRemocion> escenariosRemocion() {
    return Arbitraries.integers().between(1, 15).flatMap(size ->
        Arbitraries.integers().between(-1000, 1000).list().ofSize
            (size).flatMap(valores ->
                Arbitraries.integers().between(0, size - 1).map(index
                    ->
                        new EscenarioRemocion(crearLista(valores), index)
                    )))
}

```

```

@Provide
Arbitrary<EscenarioAgregarConCache>
escenariosAgregarConCacheNoVacía() {
    return Arbitraries.integers().between(1, 15).flatMap(size ->
        Arbitraries.integers().between(-1000, 1000).list().ofSize
            (size).flatMap(valores ->
                Arbitraries.integers().between(0, size - 1).flatMap(
                    indexARemover ->
                        Arbitraries.integers().between(-1000, 1000).map(
                            nuevoValor -> {
                                NodeCachingLinkedList ncl = crearLista(
                                    valores);
                                ncl.removeIndex(indexARemover);
                                return new EscenarioAgregarConCache(ncl,
                                    nuevoValor);
                            }
                        )
                    )
            )
    )
}

```

```
        })))));  
    }
```

El uso de `flatMap` es necesario porque el rango del índice depende del tamaño de la lista: `Arbitraries.integers().between(0, size - 1)` solo se puede armar después de haber generado `size`.

2. Ejercicio 2: Point

2.1. Contrato equals/hashCode

El contrato definido por `Object` es:

- Si `a.equals(b)`, entonces `a.hashCode() == b.hashCode()`. *Esta es la regla obligatoria*: si se viola, las estructuras basadas en hashing (`HashMap`, `HashSet`) dejan de funcionar correctamente.
- La inversa no es obligatoria: dos objetos pueden tener el mismo `hashCode` sin ser iguales (*colision*); es inevitable porque el hash es un `int` con rango finito.

2.2. Implementacion

```
@Override  
public boolean equals(Object obj) {  
    if (this == obj) return true;  
    if (obj == null || getClass() != obj.getClass()) return false  
    ;  
    Point other = (Point) obj;  
    return Float.compare(this.x, other.x) == 0  
        && Float.compare(this.y, other.y) == 0;  
}  
  
@Override  
public int hashCode() {  
    int result = Float.floatToIntBits(x);  
    result = 31 * result + Float.floatToIntBits(y);  
    return result;  
}
```

Por que `Float.compare` en `equals`. El operador `==` tiene problemas con los valores especiales del estándar IEEE 754: `NaN == NaN` es `false`, lo que rompería la reflexividad de `equals`. `Float.compare` trata correctamente `NaN` y la distinción entre `-0.0f` y `+0.0f`.

Por que Float.floatToIntBits en hashCode. Convierte el float a su representacion binaria como int. Asi, dos float iguales segun Float.compare producen el mismo int (y por lo tanto el mismo hash). La combinacion `31 * result + ...` es la formula clasica para hashear campos multiples con buena distribucion.

2.3. Propiedad 1: contrato equals/hashCode

```
@Property(tries = 250)
void puntosIgualesDebenTenerMismoHashCode(@ForAll("puntos") Point
    punto) {
    Point mismoPunto = new Point(punto.getX(), punto.getY());
    assertTrue(punto.equals(mismoPunto));
    assertEquals(punto.hashCode(), mismoPunto.hashCode());
}

@Provide
Arbitrary<Point> puntos() {
    return Combinators.combine(coordenadas(), coordenadas()).as(
        Point::new);
}

@Provide
Arbitrary<Float> coordenadas() {
    return Arbitraries.floats().between(-10_000f, 10_000f);
}
```

El generador construye puntos con coordenadas en $[-10^4, 10^4]$, rango razonable para ejercitar la combinacion de bits sin caer en valores patologicos como NaN o $\pm\infty$.

2.4. Propiedad 2: distancia sobre recta horizontal

Si $p_1 = (x_1, y)$ y $p_2 = (x_2, y)$, entonces $distance(p_1, p_2) = |x_2 - x_1|$, porque la ordenada se cancela en la formula euclidea. Esta es una propiedad geometrica *independiente* de los valores concretos: vale para todo x_1, x_2, y finitos.

```
@Property(tries = 250)
void distanciaEnRectaParalelaAlEjeXEsDiferenciaDeAbscisas(
    @ForAll("coordenadas") float x1,
    @ForAll("coordenadas") float x2,
    @ForAll("coordenadas") float y) {
    Point p1 = new Point(x1, y);
    Point p2 = new Point(x2, y);
}
```



```

    double distanciaEsperada = Math.abs((double) (x2 - x1));
    double distanciaReal = p1.distanceTo(p2);
    assertEquals(distanciaEsperada, distanciaReal, 1.0e-6);
}

```

Sobre la tolerancia. `distanceTo` usa `Math.sqrt(Math.pow(...))`, que introduce errores de redondeo. Por eso se usa `assertEquals` con tolerancia 10^{-6} . Sin tolerancia, la propiedad fallaría por ruido numérico aun siendo lógicamente correcta.

3. Ejercicio 3: Date

3.1. Constructor y `repOk()`

El constructor valida la fecha antes de inicializar los campos, y `repOk()` reusa la misma lógica:

```

public Date(int d, int m, int y) throws IllegalArgumentException
{
    if (!isValidDate(d, m, y)) throw new IllegalArgumentException
        ("invalid date");
    this.day = d; this.month = m; this.year = y;
    assert repOk();
}

public boolean repOk() { return isValidDate(day, month, year); }

private static boolean isValidDate(int d, int m, int y) {
    if (y < 1900) return false;
    if (m < 1 || m > 12) return false;
    if (d < 1) return false;
    return d <= daysInMonth(m, y);
}

private static int daysInMonth(int m, int y) {
    switch (m) {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            return 31;
        case 4: case 6: case 9: case 11: return 30;
        case 2: return leap(y) ? 29 : 28;
        default: return -1;
    }
}

```

Por que reusar. Tener *una sola* implementacion de `daysInMonth` y de la validacion evita que constructor y `addDays` se desincronicen. Cualquier cambio (eg. admitir anos < 1900) se hace en un solo lugar.

3.2. `addDays(when, days)`

Estrategia: avanzar por bloques de mes en vez de dia por dia. Para `days` del orden de miles, una iteracion por dia seria innecesariamente lenta.

```
public Date addDays(Date when, int days) {
    if (when == null) throw new IllegalArgumentException("date
        cannot be null");
    if (days < 0)      throw new IllegalArgumentException("days
        must be non-negative");

    int d = when.day, m = when.month, y = when.year;
    int remainingDays = days;

    while (remainingDays > 0) {
        int daysInCurrentMonth = daysInMonth(m, y);
        int daysLeftInMonth    = daysInCurrentMonth - d;

        if (remainingDays <= daysLeftInMonth) {
            d = d + remainingDays;
            remainingDays = 0;
        } else {
            remainingDays = remainingDays - (daysLeftInMonth + 1)
                ;
            d = 1;
            m = m + 1;
            if (m > 12) { m = 1; y = y + 1; }
        }
    }
    return new Date(d, m, y);
}
```

Invariantes del algoritmo:

- `(d, m, y)` siempre representa una fecha *valida*.
- Despues de cada iteracion, la suma “dias ya consumidos + `remainingDays`” es igual a

days.

El + 1 en el caso “no cabe”. Cuando `remainingDays > daysLeftInMonth` se consumen `daysLeftInMonth` dias para llegar al ultimo dia del mes y *uno mas* para pasar al dia 1 del mes siguiente; de ahi (`daysLeftInMonth + 1`).

Por que reconstruir con `new Date(d, m, y)` al final. El constructor valida. Si por algun bug la fecha intermedia quedara mal, la excepcion lo expondria de inmediato.

3.3. Propiedad: `addDays` siempre devuelve una fecha valida

```
@Property(tries = 250)
void addDaysSiempreDevuelveFechaValida(
    @ForAll("fechasValidas") Date fecha,
    @ForAll("diasNoNegativos") int dias) {
    Date receptor = new Date(1, 1, 1900);
    Date resultado = receptor.addDays(fecha, dias);
    assertNotNull(resultado);
    assertTrue(resultado.repOk());
}
```

Generadores.

- `fechasValidas` (`@Provide`) usa `flatMap` para que el rango del dia dependa del mes y del ano:

```
Arbitraries.integers().between(1900, 2400).flatMap(year ->
    Arbitraries.integers().between(1, 12).flatMap(month ->
        Arbitraries.integers().between(1, daysInMonth(month, year)
        )
        .map(day -> new Date(day, month, year)))));
```

Sin este encadenamiento se generarian cosas como 31/04/2024, que el constructor rechazaria.

- `diasNoNegativos`: enteros en `[0, 5000]`. El rango es lo suficientemente amplio como para forzar saltos de varios anos (5000 dias $\approx$ 13.7 anos) sin volver la suite lenta.

Sobre el receptor. `addDays` es un metodo de instancia. Se llama sobre un `new Date(1, 1, 1900)` arbitrario; el receptor no se usa en el calculo (todo se basa en `when`), asi que cualquier instancia valida sirve para invocarlo.

Conclusiones

- El testing basado en propiedades cambia el foco: en vez de pensar en *ejemplos*, se enuncia el comportamiento *en general*. El framework se encarga de buscar entradas que rompan la propiedad.
- Las propiedades mas utiles son las que capturan *invariantes* (e.g. “**repOK** se preserva”), *contratos* (**equals/hashCode**) y *relaciones algebraicas o geometricas* (distancia sobre recta horizontal).
- Los **generadores** son tan importantes como las propiedades: tienen que producir entradas validas y representativas. **flatMap** permite construir generadores *dependientes*, donde un parametro depende de otro (e.g. $\text{dia} \leq \text{dias del mes}$).
- Un buen **repOK** no es solo documentacion: *ejecuta* el invariante y lo hace verificable por las propiedades.